EXPERIMENT NO: 6

Student Name: Bhupesh Rawat
UID: 23BCS11754
Branch: BE-CSE
Section/Group: KRG-1A
Semester: 6th
Date of Performance: 26/02/2026
Subject Name: System Design
Subject Code: 23CSH-314

Aim

To design a highly scalable video streaming platform similar to YouTube that supports heavy video uploading, adaptive bitrate streaming, and real-time social interactions with high availability and low latency.

Objectives

- Understand the architecture of a massive-scale media streaming system.
- Identify functional requirements such as direct-to-cloud uploads and personalized feeds.
- Identify non-functional requirements including CDN caching, high throughput, and latency constraints.
- Analyze the decoupling of synchronous API requests using event-driven stream processing.
- Design REST APIs for video management, search, and user interactions.

Procedure

1. Studied real-world video streaming architectures and content delivery networks.
2. Identified core entities such as Users, Videos, Interactions, and View Histories.
3. Analyzed the Pre-Signed URL pattern to bypass API Gateway bottlenecks for large file uploads.
4. Collected functional and non-functional requirements for media delivery.
5. Designed APIs for video processing, metadata retrieval, and social engagement.
6. Evaluated streaming protocols (HLS/DASH) and separated thumbnail storage for optimization.
7. Analyzed stream processing (Apache Flink/Kafka) for real-time, high-volume view aggregation.

Functional Requirements

1. System should allow users to register, log in, and manage accounts.
2. System should support uploading large raw video files efficiently.
3. System should stream video smoothly using Adaptive Bitrate Streaming based on network conditions.
4. System should support real-time social interactions (likes, dislikes, comments).
5. System should allow users to search for videos by title or metadata.

6. System should generate a personalized home page feed based on watch history.

Core Entities of the System

- Users
- Videos
- Interactions (Likes/Dislikes)
- Comments
- ViewHistory

API Design

1. Authentication API

A. **Register**: POST /api/v1/auth/register
B. **Login**: POST /api/v1/auth/login

2. Video & Feed API

A. **Initialize Upload**: POST /api/v1/videos/upload/init (Returns S3 Pre-Signed URL)
B. **Get Video Metadata**: GET /api/v1/videos/{video_id}
C. **Get Personalized Feed**: GET /api/v1/feed (Pagination Supported)
D. **Search Videos**: GET /api/v1/search?q={query}

3. Interaction API

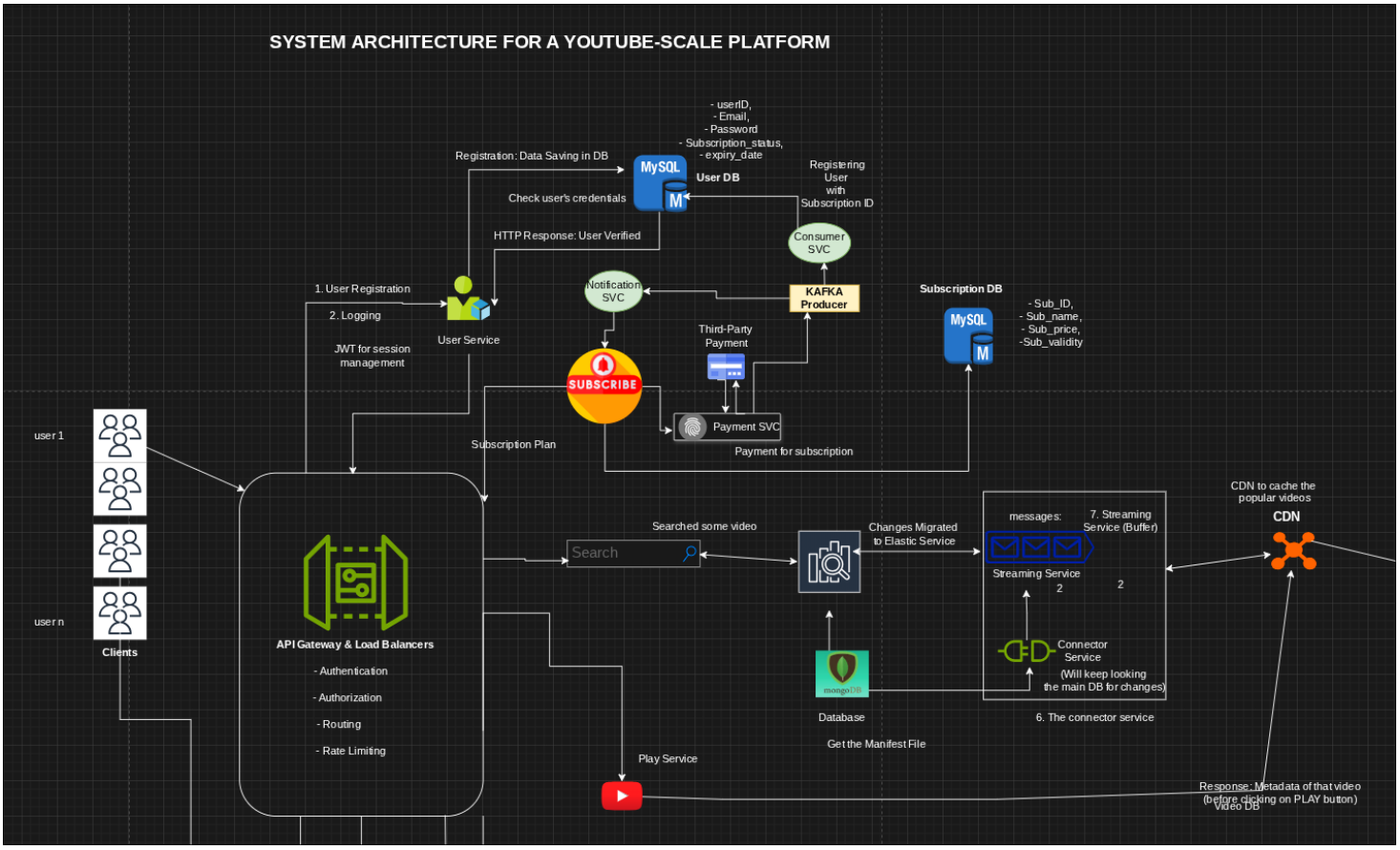
A. **Like Video**: POST /api/v1/videos/{video_id}/like
B. **Add Comment**: POST /api/v1/videos/{video_id}/comments
C. **Get Comments**: GET /api/v1/videos/{video_id}/comments

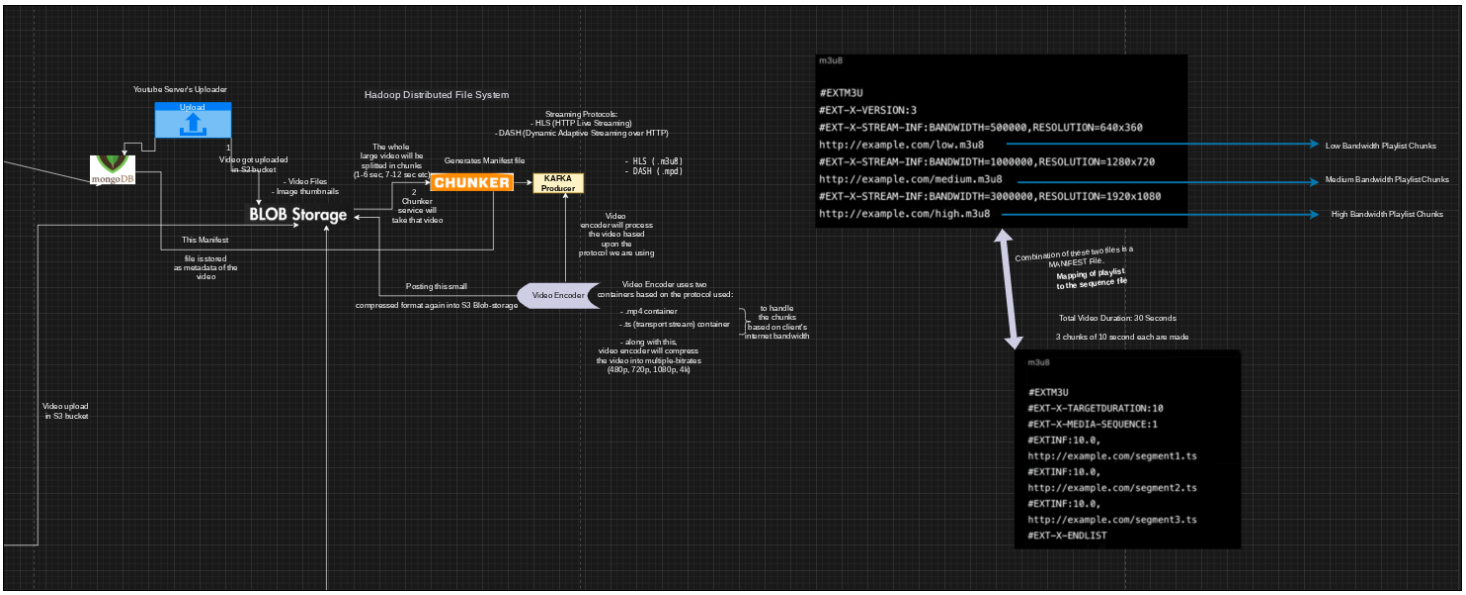
Non-Functional Requirements

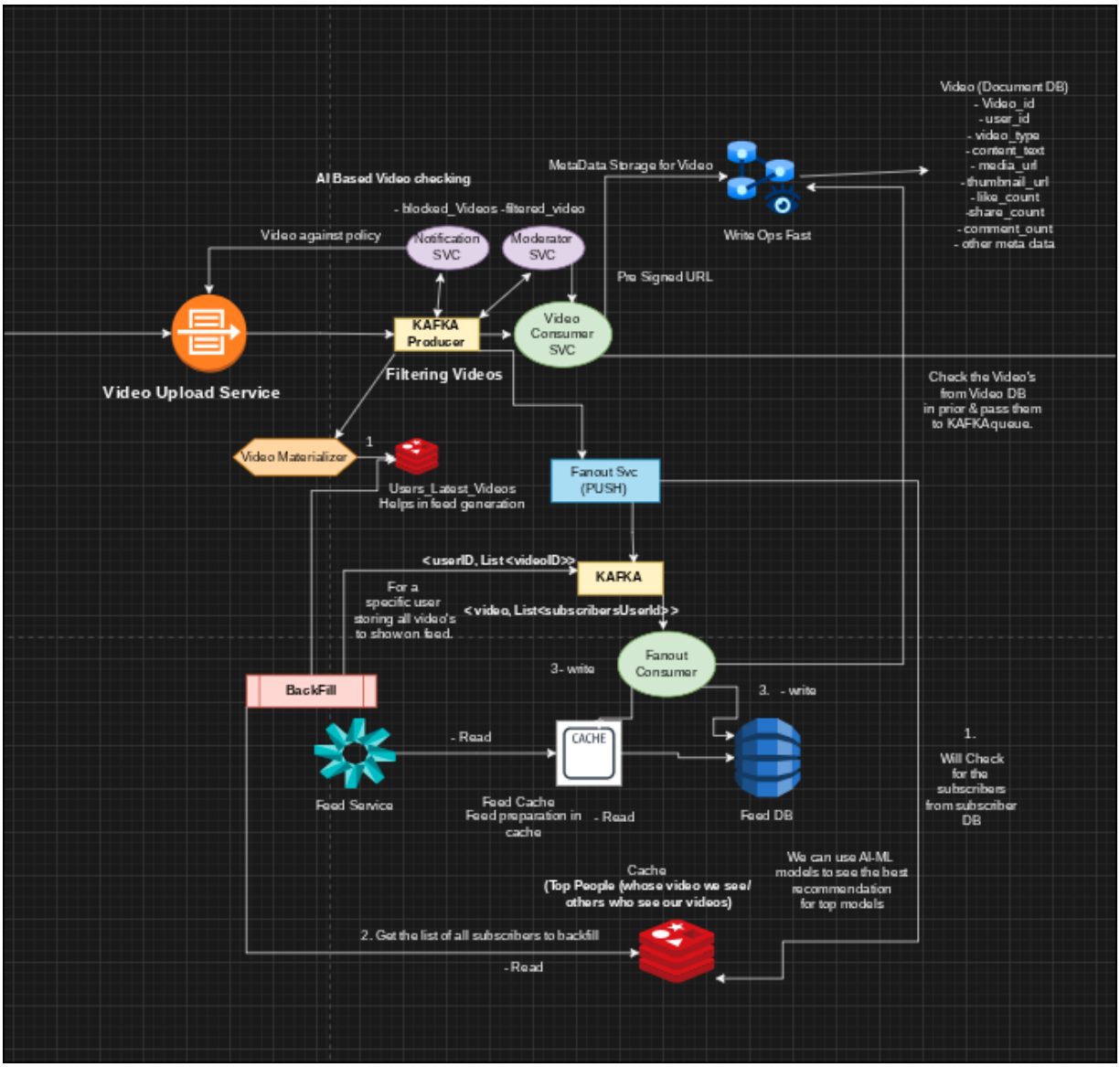
1. System must handle extremely high throughput for concurrent uploads and millions of simultaneous views.
2. High Availability must be prioritized, ensuring no single point of failure.
3. System should follow an Eventual Consistency model for social metrics (view counts, likes) to optimize database write loads.
4. Video delivery latency should be minimized via Edge Caching and a Global CDN.
5. Transcoding pipeline must be horizontally scalable to handle viral upload spikes.

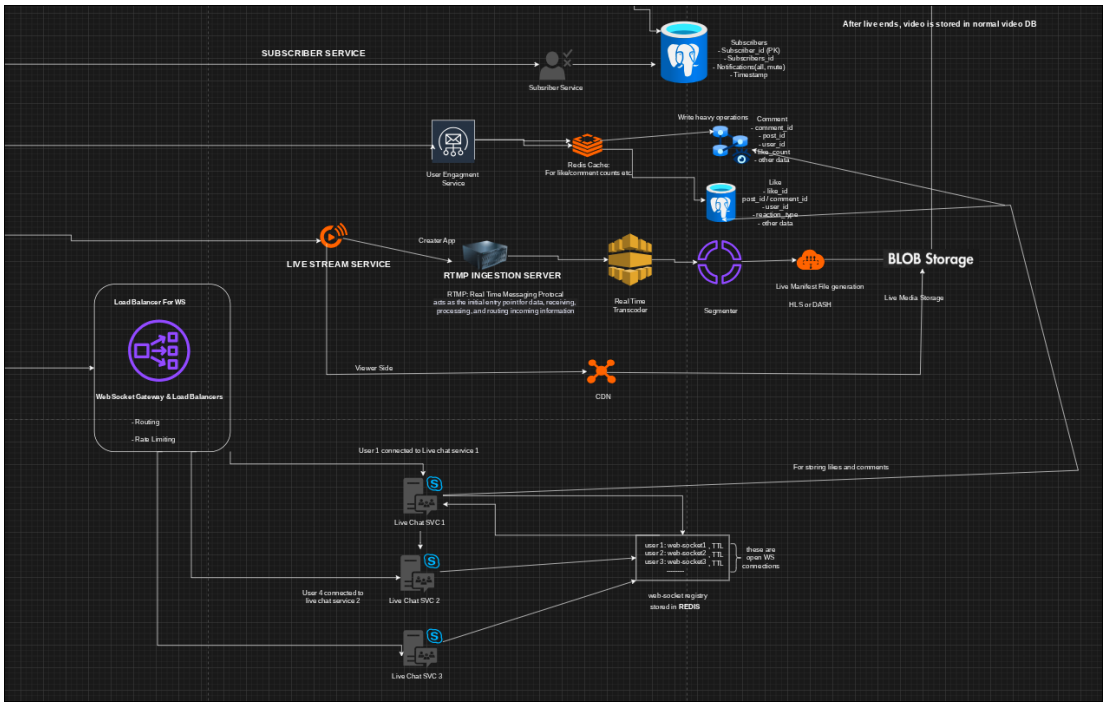
High Level Design (HLD)

The system consists of Client Applications, an API Gateway, and specialized microservices (Auth, Upload, Video Metadata, Feed, and Interactions). Heavy uploads bypass the backend via Pre-Signed URLs directly to Object Storage. Asynchronous transcoding pipelines process videos into HLS/DASH segments, which are delivered globally via a CDN. High-volume interactions are managed using Redis (Likes), NoSQL clusters (Comments), and a Kafka-to-Apache Flink stream processing pipeline for accurate view aggregation.


SYSTEM ARCHITECTURE FOR A YOUTUBE-SCALE PLATFORM
- userID,
- Email,
- Password
- Subscription_status,
- expiry_date
Registration: Data Saving in DB
MySQL
User DB
Registering User with Subscription ID
Check user's credentials
HTTP Response: User Verified
Consumer SVC
1. User Registration
2. Logging
User Service
JWT for session management
Notification SVC
KAFKA Producer
Subscription DB
- Sub_ID,
- Sub_name,
- Sub_price,
-Sub_validity
Third-Party Payment
SUBSCRIBE
Payment SVC
Payment for subscription
Subscription Plan
user 1
user n
Clients
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Searched some video
Search
Changes Migrated to Elastic Service
messages:
7. Streaming Service (Buffer)
Streaming Service
2
2
CDN to cache the popular videos
CDN
Connector Service
(Will keep looking the main DB for changes)
Database
6. The connector service
Get the Manifest File
Play Service
Response: Metadata of that video (before clicking on PLAY button)
Video DB


Youtube Server's Uploader
Upload
mongoDB
Hadoop Distributed File System
1
Video got uploaded in S3 bucket
- Video Files
- Image thumbnails
BLOB Storage
This Manifest
file is stored as metadata of the video
The whole large video will be splitted in chunks (1-6 sec, 7-12 sec etc)
Generates Manifest file
CHUNKER
KAFKA Producer
2
Chunker service will take that video
Streaming Protocols:
- HLS (HTTP Live Streaming)
- DASH (Dynamic Adaptive Streaming over HTTP)
- HLS (.m3u8)
- DASH (.mpd)
Video encoder will process the video based upon the protocol we are using
Posting this small compressed format again into S3 Blob-storage
Video Encoder
Video Encoder uses two containers based on the protocol used:
- .mp4 container
- .ts (transport stream) container
to handle the chunks based on client's internet bandwidth
- along with this, video encoder will compress the video into multiple bitrates (480p, 720p, 1080p, 4k)
Video upload in S3 bucket
m3u8
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-STREAM-INF:BANDWIDTH=500000,RESOLUTION=640x360
http://example.com/low.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1000000,RESOLUTION=1280x720
http://example.com/medium.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1920x1080
http://example.com/high.m3u8
Low Bandwidth Playlist Chunks
Medium Bandwidth Playlist Chunks
High Bandwidth Playlist Chunks
Combination of these two files is a MANIFEST file
Mapping of playlist to the sequence file
Total Video Duration: 30 Seconds
3 chunks of 10 second each are made
m3u8
#EXTM3U
#EXT-X-TARGETDURATION:10
#EXT-X-MEDIA-SEQUENCE:1
#EXTINF:10.0,
http://example.com/segment1.ts
#EXTINF:10.0,
http://example.com/segment2.ts
#EXTINF:10.0,
http://example.com/segment3.ts
#EXT-X-ENDLIST


Video (Document DB)
- Video_id
- user_id
- video_type
- content_text
- media_url
- thumbnail_url
- like_count
-share_count
- comment_count
- other meta data
MetaData Storage for Video
Write Ops Fast
AI Based Video checking
- blocked_Videos -filtered_video
Video against policy
Notification SVC
Moderator SVC
Pre Signed URL
KAFKA Producer
Video Consumer SVC
Video Upload Service
Filtering Videos
Check the Video's from Video DB in prior & pass them to KAFKAqueue.
Video Materializer
1
Users_Latest_Videos
Helps in feed generation
Fanout Svc (PUSH)
<userID, List<videoID>>
KAFKA
For a specific user storing all video's to showon feed.
<video, List<subscribersUserId>>
3- write
Fanout Consumer
3. - write
BackFill
- Read
CACHE
Feed Service
Feed Cache
Feed preparation in cache
- Read
Feed DB
1.
Will Check for the subscribers from subscriber DB
Cache
(Top People (whose video we see/ others who see our videos)
We can use AI-ML models to see the best recommendation for top models
2. Get the list of all subscribers to backfill
- Read


After live ends, video is stored in normal video DB
SUBSCRIBER SERVICE
Subscriber Service
- Subscribers
-Subscriber_id (PK)
- Subscribers_id
- Notifications(all, mute)
- Timestamp
Write heavy operations
Comment
- comment_id
- post_id
- user_id
- like_count
- other data
User Engagement Service
Redis Cache:
For likes/comment counts etc
Like
-like_id
post_id/comment_id
-user_id
- liked(t/f, false)
- other data
LIVE STREAM SERVICE
Creator App
RTMP INGESTION SERVER
RTMP: Real Time Messaging Protocol acts as the initial entry point for data, receiving, processing, and routing incoming information
Real Time Transcoder
Segmenter
Live Manifest File generation
HLS or DASH
BLOB Storage
Live Media Storage
Load Balancer For WS
Web Socket Gateway & Load Balancers
- Routing
- Rate Limiting
Viewer Side
CDN
User 1 connected to Live chat service 1
For storing likes and comments
Live Chat SVC 1
user 1: web-socket1 , TTL
user 2: web-socket2 , TTL
user 3: web-socket3 , TTL
these are open WS connections
User 4 connected to live chat service 2
Live Chat SVC 2
web-socket registry stored in REDIS
Live Chat SVC 3

Outcome

- Designed a highly scalable, event-driven video streaming architecture.
- Identified critical functional and non-functional requirements for media delivery.
- Designed REST APIs and decoupled asynchronous processing pipelines.
- Understood the application of CDNs, adaptive bitrate streaming, and stream processing for massive-scale systems.